IST 736 · UNDERSTANDING LLMS · WEEKS 2–7

Midterm Study Guide

Everything you need to answer the Spring 2026 midterm: how text becomes tokens, how words become vectors, how a neural network turns vectors into next-token probabilities, how transformers build context, and how training and prompting shape what a model does.

What this guide covers

	Topic area	Key concepts	Midterm	Slides	Page
A	**Tokenization**	Vocabulary size trade-offs; subword tokenization (BPE, WordPiece) and out-of-vocabulary words; how tokenizers handle numbers; why the same text costs more tokens in some languages	Q1–Q5	Week 2	**3**
B	**Word embeddings & similarity**	Co-occurrence matrices; cosine similarity; word2vec positive vs. negative examples; evaluating embeddings with analogies; word vs. sentence embeddings	Q6–Q10	Week 3	**8**
C	**Neural language models**	Logits, softmax, cross-entropy loss; gradient descent and learning rate; softmax regression as a language model; why nonlinear activations matter; counting parameters	Q11–Q17	Week 4	**13**
D	**Transformers & attention**	Query, key, value; static vs. contextual embeddings; multi-head attention; the language modeling head; positional embeddings; the residual stream	Q18–Q23	Week 5	**19**
E	**Shaping model behavior**	Pretraining, SFT, and RLHF; SFT data vs. preference data; in-context (few-shot) learning; overfitting and early stopping; scaling laws; the KL penalty; the policy in RL	Q24–Q30	Weeks 5–7	**23**
	The big picture · one prediction traced through every topic				**2**
	Formula sheet & answer key · every equation and hand-calculation value on one page				**30**
	Glossary · every highlighted term, A–Z				**31**
	AI use statement				**34**

How to use this guide

1. Read **the big picture** first. The five topics are five stages of the same pipeline, so each one makes more sense once you see where it sits.
2. In each topic, read the core ideas, then **redo every worked example on paper without looking**. The exam is closed-book, pen-and-paper.
3. End each topic with its **midterm check**, which explains why the right answer is right and why the other options are wrong.
4. The night before: the formula sheet, then the glossary.

How to read the page

highlighted term: an important technical term, defined in the glossary.

✎ **WORKED EXAMPLE**: a calculation with every step shown.

⚠ **EXAM TRAP**: a tempting wrong answer, and why it is wrong.

KEY IDEA: the one sentence to remember.

BEYOND THE SLIDES: background from general knowledge, marked so you know it wasn't in lecture.

THE BIG PICTURE · READ THIS FIRST

One prediction, traced through all five topics

A language model does one thing: it gives every possible next token a probability. Here **“The cat sat on the”** goes through GPT-4o’s pipeline (real o200k_base token IDs). The letter on each row is the topic that covers it.

	STAGE	WHAT HAPPENS	WHAT EXISTS NOW
·	**Raw text**	A string of characters, stored as UTF-8 bytes.	`"The cat sat on the"`
A	**Tokenizer**	Byte-level BPE splits the text into “The”·“ cat”·“ sat”·“ on”·“ the”. Each token’s ID is its row number in the vocabulary.	`[976, 9059, 10139, 402, 290]`
B	**Embedding lookup**	Each ID selects one row of the embedding matrix E (|V| × d). These are dense vectors, and similar words have similar vectors.	`5 × d`
D	**+ Position**	A positional embedding is added to each token vector, so word order is visible to the model.	`5 × d`
D	**Transformer blocks ×L**	In every block, attention (query·key → weights → sum of values) mixes in information from earlier tokens, and an FFN processes each token. Both *add* their output to the residual stream. The output is contextual embeddings.	`5 × d → 5 × d`
D	**LM head**	The last token’s final hidden state is multiplied by E^{T}, giving one score (logit) per vocabulary token.	`1 × d → 1 × |V|`
C	**Softmax**	Logits become a probability distribution: all positive, summing to 1. (The numbers here are illustrative, taken from the Q12 example.)	`P(" mat") = 0.58 …`
C	**Cross-entropy loss**	*Training only:* compare the prediction with the true next token. Loss = −log P(correct).	`−ln 0.58 = 0.55`
C	**Gradient descent**	*Training only:* backpropagation computes gradients, and every weight moves a small step (the learning rate) downhill.	`w ← w − η∇L`

E Who changes the model, and how: which data the loss is computed on, or no weight change at all

STAGE 1	STAGE 2	STAGE 3	STAGE 4 (MODERN)
Pretraining	**SFT**	**RLHF / preference**	**RLVR**
Next-token prediction on trillions of tokens of web text, code and books. Learns language and world knowledge.	The same loss on instruction → answer pairs. Learns to follow instructions and chat.	A reward model learns from chosen vs. rejected answers, and RL (PPO) pushes the policy toward high reward, with a KL penalty.	RL with automatically checkable rewards (math, code). Learns multi-step reasoning.

DEVELOPERS CHANGE THE WEIGHTS

Stages 1–4 happen before release and change the model permanently, for every user. Throughout training, **validation loss** is watched to catch overfitting, and **scaling laws** predict how loss improves with model size, data and compute.

USERS CHANGE THE CONTEXT

System prompts, few-shot examples (in-context learning), RAG and sampling settings all act on a frozen model. Nothing is learned permanently, and the effect lasts only for that conversation.

TOPIC A · WEEK 2 SLIDES · MIDTERM Q1–Q5

A Tokenization

YOU SHOULD BE ABLE TO

- Explain what goes wrong when a vocabulary is too large or too small
- Trace the BPE merge algorithm by hand
- Explain how subword tokenizers remove the OOV problem
- Predict how GPT-4o splits a number into tokens
- Explain why non-English text costs more tokens
- Tell word types from word instances

A.1 Tokens, types and the vocabulary

A **tokenizer** splits text into **tokens**, which can be words or pieces of words (**subwords**). The set of all distinct tokens the tokenizer knows is the **vocabulary** V, and $|V|$ is the **vocabulary size**. Every token has an integer **token ID**, which is simply its row number in the model's embedding matrix.

When counting words, separate the **type** (a distinct vocabulary item) from the **instance** (each occurrence in running text). *"They picnicked by the pool, then lay back on the grass and looked at the stars."* has **16 instances but 14 types**, because "the" occurs three times (ignoring punctuation).

Why LLMs don't simply use whole words as tokens

- **There are too many words.** Larger corpora keep producing new types: Shakespeare has 31K types in 884K instances, while Google N-grams has more than 13 million types in 1 trillion instances.
- **Some scripts have no spaces.** Chinese, Japanese and Thai don't mark word boundaries. 姚明进入总决赛 can reasonably be split into 3, 5 or 7 "words".
- **Unknown words are unavoidable.** However big a word list is, a new word will eventually appear. This is the **out-of-vocabulary (OOV)** problem. Word-level systems map such words to a single **[UNK]** token and lose them.

A.2 Vocabulary size is a trade-off Q1 Q2

	Vocabulary too large (millions)	Vocabulary too small (a few hundred)
What happens	Many tokens are rare and appear only a few times, or never, in the training data.	Most words must be split into many short pieces, sometimes down to single characters or bytes.
Main problem	**Statistical inefficiency.** Rare tokens get too little training signal, so their embeddings and output probabilities are poorly estimated.	**Longer sequences.** More tokens per sentence means more computation. Self-attention compares every token with every other, so its cost grows with the *square* of the sequence length, and less text fits in the context window.
Also	The embedding matrix ($\|V\| \times d$) and the output layer (one logit per token) both grow with $\|V\|$, so there are more parameters and the softmax is slower.	Each token carries little meaning by itself, so the model has to assemble meaning across many positions.

	Vocabulary too large (millions)	Vocabulary too small (a few hundred)
Not a reason	It does not change the number of attention heads or layers. Those are separate design choices.	It does not force more attention heads or more parameters per layer, and it does not make the model “underfit by definition”.

KEY IDEA

Vocabulary size affects three things: the input end (embedding matrix), the output end (LM head and softmax over |V|), and how many tokens a text becomes. Modern LLMs settle between about **50,000 and 200,000** tokens. GPT-4o’s tokenizer, **o200k_base**, has about 200K.

A.3 Subword tokenization: BPE and WordPiece Q3

Subword tokenization keeps frequent words as single tokens and builds rare words from pieces. GPT-4o (verified with its tokenizer) splits `tokenization` into `token` `ization`, and `unbelievably` into `un` `bel` `ievably`. Every subword algorithm has two parts:

- a **token learner**, which reads a training corpus and induces the vocabulary, and
- a **token segmenter** (encoder), which splits new text using that vocabulary.

Byte-Pair Encoding (BPE) learns its vocabulary by repeated merging. Start with every individual symbol. Then, k times: find the **most frequent adjacent pair** (t_L, t_R), add the merged token $t_L t_R$ to the vocabulary, and replace every occurrence of the pair in the corpus.

✎ WORKED EXAMPLE · BPE MERGES (FROM THE WEEK 2 SLIDES)

Corpus: `A B D C A B E C A B` Starting vocabulary: {A, B, C, D, E}. Do 2 merges.

1. List the adjacent pairs and count them: AB ×**3**, CA ×2, BD ×1, DC ×1, BE ×1, EC ×1.
2. Merge the most frequent pair, AB. The corpus becomes `AB D C AB E C AB` and the vocabulary becomes {A, B, C, D, E, **AB**}.
3. Recount on the *new* corpus: (C, AB) ×**2**, (AB, D) ×1, (D, C) ×1, (AB, E) ×1, (E, C) ×1.
4. Merge C + AB. The corpus becomes `AB D CAB E CAB` and the vocabulary becomes {A, B, C, D, E, AB, **CAB**}.

Result: frequent sequences turn into longer tokens. Running k merges adds k tokens, so **the number of merges sets the vocabulary size**.

- **Pre-tokenization**: before BPE runs, regular expressions split text at spaces and punctuation, and merges never cross those boundaries. The leading space stays attached to the word (“ cat”), so `Anyhow` and ` anyhow` are different tokens.
- **Byte-level BPE**: modern LLMs run BPE on the **UTF-8 bytes** of the text, not on characters. There are only 256 possible bytes and all of them are in the base vocabulary, so **any string can be encoded and nothing is ever unknown**. There are no OOV words and no [UNK]. Along the way, BPE rediscovers the common 2- and 3-byte UTF-8 sequences.

BEYOND THE SLIDES · WORDPIECE

WordPiece (used by BERT) builds its vocabulary by merging, like BPE, but chooses merges differently. Instead of raw pair frequency, it scores each pair by how much the merge improves the likelihood of the training data, roughly `count(AB) / (count(A) × count(B))`. This favors pairs whose parts rarely occur apart. It marks word-internal pieces with ##: "unaffable" → `un` `##aff` `##able`. For the exam, BPE and WordPiece solve OOV the same way. (The slides' other main algorithm is the Unigram LM tokenizer, often called "SentencePiece".)

KEY IDEA · HOW SUBWORDS SOLVE OOV

A word the model has never seen is **represented as a combination of smaller pieces that are already in the vocabulary**. In the worst case those pieces are single characters or bytes. The vocabulary is fixed after training; nothing is added at inference time.

A.4 Unicode and UTF-8: why bytes matter

Unicode gives each of its 150,000+ characters (in 168 scripts) a unique **code point**, written like U+0061 for "a". Files store an *encoding* of those code points, and almost the whole web uses **UTF-8**. UTF-8 is a variable-length encoding: ASCII characters take 1 byte, most European, Middle Eastern and African characters take 2, most Chinese, Japanese and Korean characters take 3, and emoji take 4.

Text	Characters	UTF-8 bytes	Bytes (hex)
cat	3	3	`63 61 74`
señor	5	6	`73 65 C3 B1 6F 72` (ñ = 2 bytes)
猫 (cat)	1	3	`E7 8C AB`
안녕 (hello)	2	6	`EC 95 88 EB 85 95`
😀	1	4	`F0 9F 98 80`

Byte-level BPE starts from these bytes. Characters that need more bytes, and are rare in the training data, begin as more pieces and get merged less often. In Python 3, `len()` counts code points, while `len(s.encode("utf-8"))` counts bytes.

A.5 How tokenizers handle numbers Q4

Numbers aren't learned like words. A pre-tokenization rule decides how digits are chunked. GPT-4o's tokenizer splits **runs of digits into chunks of up to 3, reading left to right**, and the space before a number becomes **its own token** (ID 220).

✎ WORKED EXAMPLE · Q4: WHICH IDS ARE THE ZIP CODE?

"The zip code for Syracuse University is 13078." → 976, 16216, 3490, 395, 108245, 4923, 382, 220, 9300, 4388, 13

1. Count the words before the number: The, zip, code, for, Syracuse, University, is. That is 7 ordinary words with their leading spaces, so the first 7 IDs (976 … 382) are those words.
2. The sentence ends in ".", so the last ID (13) is the period.
3. That leaves 220, 9300 and 4388 for " 13078". The space before a number is a separate token, so 220 is " ".

4 Chunk 13078 into groups of 3 from the left: `130` `78`. That is two tokens.

Answer: **9300 ("130") and 4388 ("78")**. Running the actual o200k_base tokenizer confirms it: `The` `zip` `code` `for` `Syracuse` `University` `is` ` ` `130` `78` `.`

More GPT-4o examples (verified): `1234567890` → `123` `456` `789` `0`. The Llama 3 tokenizer on the slide also produces 4 tokens. `In 2026 I paid $1,307.85` → `In` ` ` `202` `6` `I` `paid` `$` `1` `,` `307` `.` `85`.

- **Why it matters:** the model never sees "2026" as a single number. It sees "202" followed by "6". The chunks don't line up with place value: 13078 → 130|78 but 1307 → 130|7. This makes arithmetic and comparing numbers harder.
- **Medical records:** popular LLMs split timestamps and sensor values into "multiple nonmeaningful tokens". Epic's CoMET model instead **discretizes** continuous lab values into bins, which hides small but sometimes important differences.

A.6 Why the same text costs more tokens in some languages Q5

BPE learns its merges from its training corpus, and LLM training data is **dominated by English**. Most vocabulary slots therefore go to English words and word pieces. English words usually become one token each, while words in other languages are **over-segmented** into several pieces. UTF-8 adds to the effect, because non-Latin scripts start at 2–4 bytes per character.

Example from the slides (GPT-4o / Tiktokenizer)	English	Other language
The same recipe sentence	18 tokens, no words split	Spanish: 33 tokens, 6 of 16 words split
"Hello how are you?"	5 tokens	Korean: 15 tokens

Consequences: higher cost, because APIs bill per token; less text fits in the context window; generation is slower; and representations are poorer. Low-resource languages can be split all the way down to single characters.

⚠ EXAM TRAP · Q5

The cause is **what the tokenizer's vocabulary covers**, not the language itself. Spanish is not "more complex" and doesn't need a bigger model (A). The difference isn't just sentence length (C). Character count alone doesn't force splits either (D): a long word that is in the vocabulary is still one token.

A.7 Other Week 2 points worth knowing

- **Why tokenize deterministically:** systems can be compared fairly, because they agree on how long a text is, and measures like perplexity assume a fixed tokenization. Any word in test data can still be segmented into known tokens.
- **Token efficiency:** YAML often uses fewer tokens than JSON, but JSON is unambiguous, while YAML depends on indentation. Hidden reasoning tokens are often billed even when you can't see them.
- **Corpora** vary by language, variety (e.g. African American English), genre, author demographics and **code switching**. Document these in a datasheet.

MIDTERM CHECK · TOPIC A

Q1	**C**	An enormous vocabulary has many rare tokens that get too little training signal (statistical inefficiency). (A) is backwards: a bigger vocabulary means *fewer* tokens per sentence. (B) and (D) are wrong because heads and layers don't depend on vocabulary size.
Q2	**A**	A tiny vocabulary splits words into many pieces, which means longer sequences and more self-attention computation. (C) is wrong: having fewer embedding parameters doesn't make a model underfit "by definition".
Q3	**B**	Rare or unseen words are built from smaller subword units that are already in the vocabulary. (A) is wrong because the vocabulary is fixed after training. (C) describes the old [UNK] workaround, not the fix.
Q4	**9300, 4388**	"13078" is split into chunks of up to 3 digits from the left: "130" + "78". ID 220 is the separate space token.
Q5	**B**	The tokenizer was trained on mostly English data, so its vocabulary covers English words better.

TOPIC B · WEEK 3 SLIDES · MIDTERM Q6–Q10

B Word embeddings & similarity

YOU SHOULD BE ABLE TO

- Fill in a word–word co-occurrence matrix for any window size
- Compute cosine similarity by hand, and explain why it beats the raw dot product
- Explain what makes a word2vec pair positive or negative
- Evaluate embeddings with analogies
- Choose between word and sentence embeddings for a task

B.1 The distributional hypothesis

A bag-of-words representation can't tell that *fast* and *quick* are synonyms, or that *apple*, *pear* and *strawberry* are related. Embeddings rest on the **distributional hypothesis**: **words that occur in the same contexts tend to have similar meanings** (Harris, 1954), or, as Firth (1957) put it, "a word is characterized by the company it keeps".

B.2 Co-occurrence (word–context) matrices Q6

A **co-occurrence matrix** (also called a term–term or word–context matrix) has one row and one column per vocabulary word, so it is |V| × |V|. Cell (*w*, *c*) counts **how many times context word *c* appears inside the window around target word *w***. The **context window** can be a whole document or a few words on each side (±4 is common). A window of 1 means just the immediate left and right neighbors. With a symmetric window the matrix is **symmetric**. It is also **sparse**, since a 50,000 × 50,000 matrix is mostly zeros. Each word's row is its (long, sparse) vector.

✎ **WORKED EXAMPLE · FILLING IN A CO-OCCURRENCE MATRIX (WINDOW = 1)**

Documents: D1 "I like NLP" · D2 "I like deep learning" · D3 "I enjoy flying". Windows never cross document boundaries.

1. List the vocabulary: I, like, enjoy, deep, learning, NLP, flying. That gives a 7 × 7 matrix.
2. Walk through each document and write down each word's neighbors:
   D1: I→{like} · like→{I, NLP} · NLP→{like}
   D2: I→{like} · like→{I, deep} · deep→{like, learning} · learning→{deep}
   D3: I→{enjoy} · enjoy→{I, flying} · flying→{enjoy}
3. Tally the adjacent pairs: (I, like) = **2** (D1 and D2) · (I, enjoy) = 1 · (like, NLP) = 1 · (like, deep) = 1 · (deep, learning) = 1 · (enjoy, flying) = 1. Every other pair is 0.

4. Fill both (*w*, *c*) and (*c*, *w*), since the matrix is symmetric:

	I	like	enjoy	deep	learning	NLP	flying
I	0	**2**	1	0	0	0	0
like	**2**	0	0	1	0	1	0
enjoy	1	0	0	0	0	0	1
deep	0	1	0	0	1	0	0
learning	0	0	0	1	0	0	0
NLP	0	1	0	0	0	0	0
flying	0	0	1	0	0	0	0

5. Check your work. Each row sum should equal the number of neighbor slots that word fills. “like” occurs twice, each time with 2 neighbors, and its row sums to 2 + 1 + 1 = 4 ✓. The whole matrix sums to 2 × (number of adjacent pairs) = 2 × 7 = 14 ✓.

✎ WORKED EXAMPLE · Q6: THE MISSING CELLS

D1 “the cat sat” · D2 “the cat ate” · D3 “the dog sat”, window = 1. Find (cat, dog) and (dog, cat).

1. Find where “cat” and “dog” occur. cat is in D1 and D2, and dog is in D3. They are never in the same document.
2. So dog is never immediately left or right of cat, which means the count is 0. By symmetry, (dog, cat) is also 0.
3. Sanity-check the dog row: the 1 (D3), sat 1 (D3), cat 0, ate 0. That is 2 neighbor slots for one occurrence ✓.

Answer: **both “?” cells are 0**. Note: by the same rule, (the, cat) = 2 (D1 and D2). The exam prints 3, which looks like a typo. It doesn’t affect the “?” cells.

B.3 Dot product, vector length and cosine similarity Q7

The **dot product** is high when two vectors have large values in the same dimensions:

$$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{N} v_i w_i \qquad |\mathbf{v}| = \sqrt{\sum_i v_i^2}$$

Problem: the raw dot product favors **long vectors**. Frequent words like *the*, *of* and *you* co-occur with everything, so they get large dot products with every word.

Cosine similarity divides by both lengths, which removes the effect of frequency. The result is the cosine of the angle between the vectors:

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}|\,|\mathbf{w}|}$$

+1 means the same direction, 0 means orthogonal (unrelated), −1 means opposite. Raw counts are never negative, so **count vectors give 0 to 1**. Learned embeddings can be negative and use the full −1 to 1 range. Length doesn’t matter: (1, 2) and (10, 20) have a cosine of exactly 1.

✎ WORKED EXAMPLE · Q7: COSINE SIMILARITY

$\mathbf{v}_1 = (1, 2, 2)$, $\mathbf{v}_2 = (2, 1, 2)$

1. Dot product: $1 \cdot 2 + 2 \cdot 1 + 2 \cdot 2 = 2 + 2 + 4 = 8$

2. Lengths: $|\mathbf{v}_1| = \sqrt{1+4+4} = \sqrt{9} = 3$ and $|\mathbf{v}_2| = \sqrt{4+1+4} = \sqrt{9} = 3$
3. Divide: $\cos = \frac{8}{3 \times 3} = \frac{8}{9}$

Answer: **8/9 ≈ 0.889**. The vectors point in nearly the same direction (angle ≈ 27°), so the two words are very similar.

✎ WORKED EXAMPLE · WHICH WORD IS CLOSER TO "INFORMATION"? (WEEK 3 SLIDES)

Counts with the context words (pie, data, computer): cherry = (442, 8, 2) · digital = (5, 1683, 1670) · information = (5, 3982, 3325)

1. **cherry · information** = 442·5 + 8·3982 + 2·3325 = 2,210 + 31,856 + 6,650 = **40,716**
2. |cherry| = √(442² + 8² + 2²) = √195,432 ≈ 442.08 · |information| = √(5² + 3982² + 3325²) = √26,911,974 ≈ 5,187.68
3. cos(cherry, information) = 40,716 / (442.08 × 5,187.68) = 40,716 / 2,293,350 ≈ **0.018** (the slide shows .017, a rounding difference)
4. **digital · information** = 5·5 + 1683·3982 + 1670·3325 = 25 + 6,701,706 + 5,552,750 = **12,254,481**; |digital| = √5,621,414 ≈ 2,370.95
5. cos(digital, information) = 12,254,481 / (2,370.95 × 5,187.68) = 12,254,481 / 12,299,730 ≈ **0.996**

Result: **digital (0.996) is far closer to information than cherry (0.018)**. Both appear near "data" and "computer". The angle between their vectors is tiny.

B.4 From sparse counts to dense embeddings

	Count vectors (co-occurrence, tf-idf)	Dense embeddings
Length	|V| (20,000–50,000)	Short, usually 50–1,000 dimensions
Values	Mostly zeros (sparse), non-negative counts	Mostly non-zero real numbers, which can be negative
Dimensions	Each one is a context word, so it can be interpreted	No clear interpretation
Methods	Counting	**Static embeddings**: word2vec (skip-gram, CBOW), GloVe, SVD/LSA. **Contextual embeddings**: BERT and GPT hidden states (Topic D)

B.5 word2vec: positive vs. negative examples Q8

word2vec uses **skip-gram with negative sampling (SGNS)**. Instead of counting, it **predicts**: it trains a binary classifier to decide whether a word *c* really appears near target word *w*. This is **self-supervised learning**, because running text supplies the labels for free.

- **Positive example**: (target, a word that **actually occurs inside the target's context window** in the corpus).
- **Negative example**: (target, a **randomly sampled "noise" word** from the vocabulary, drawn according to weighted unigram frequency). It is a pair that was not observed in the text. There are k negatives for every positive.

Classifier: $P(+ \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + e^{-\mathbf{c} \cdot \mathbf{w}}}$, and $P(- \mid w, c) = 1 - P(+ \mid w, c)$. Training uses stochastic gradient descent to raise the dot products of positive pairs and lower those of negative pairs:

$$L = -\left[\log \sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) + \sum_{i=1}^{k} \log \sigma(-\mathbf{c}_{neg_i} \cdot \mathbf{w})\right]$$

✎ EXAMPLE · TRAINING PAIRS FOR "APRICOT"

"… lemon, a [**tablespoon of apricot jam, a**] pinch …" (window ±2, $k = 2$)

Positive (+)	Negative (–), 2 per positive	
apricot, tablespoon	apricot, aardvark	apricot, seven
apricot, of	apricot, my	apricot, forever
apricot, jam	apricot, where	apricot, dear
apricot, a	apricot, coaxial	apricot, if

SGNS learns **two** matrices: target embeddings W and context embeddings C, each |V| × d. A word is usually represented as $\mathbf{w}_i + \mathbf{c}_i$. After training, the classifier is thrown away and only the embeddings are kept.

B.6 Evaluating embeddings with analogies Q9

The standard *intrinsic* test is the **analogy test**: if relationships are captured as consistent directions in the vector space, then **vector arithmetic** should solve analogies. Compute **king − man + woman** and find the nearest word by cosine similarity, excluding the three input words. It should be **queen**. Likewise, Paris : France :: Tokyo : **Japan**. Mikolov's test set has semantic relations (capital–country, currency, city–state, man–woman) and syntactic ones (adjective → adverb, opposite, comparative, superlative, past tense, plural).

- *Beyond the slides:* another intrinsic test correlates cosine scores with human word-similarity ratings (e.g. WordSim-353, SimLex-999). *Extrinsic* evaluation measures performance on a downstream task such as classification, but that is indirect.
- **Analogies also reveal bias.** man : computer programmer :: woman : *homemaker*, and father : doctor :: mother : *nurse*. Embeddings absorb stereotypes from the text they are trained on, which can cause **allocational harm** (e.g. in hiring searches). Historical embeddings trace changes in meaning and bias over the decades.

B.7 Word vs. sentence embeddings Q10

A **word embedding** is one vector per word. A **sentence embedding** is one vector for a whole sentence or document. You could build one by **pooling** the word vectors:

✎ WORKED EXAMPLE · MEAN VS. MAX POOLING (WEEK 4 SLIDES)

4 tokens with 3-dimensional embeddings: [1, 2, 3], [2, 2, 3], [3, 2, 3], [4, 2, 3]

1. **Mean pooling** averages each dimension: [(1+2+3+4)/4, (2+2+2+2)/4, (3+3+3+3)/4] = **[2.5, 2, 3]**
2. **Max pooling** takes the largest value in each dimension: **[4, 2, 3]**

Averaging static word vectors makes a poor sentence embedding. It throws away word order ("dog bit man" = "man bit dog"), gives each word the same vector in every context, lets frequent function words dilute the meaning, and loses negation ("not good" ends up close to "good").

Sentence-BERT (SBERT) fixes this. It runs BERT/RoBERTa, applies mean pooling, and **fine-tunes the result in a Siamese (twin) network** on sentence pairs labeled entailment, contradiction or neutral (SNLI 570K pairs plus MultiNLI 430K). The result is sentence vectors whose **cosine similarity reflects meaning**. That is exactly what clustering needs. BERTopic (SBERT + UMAP + HDBSCAN) uses it, and Anthropic analyzed user conversations with text embeddings plus k-means.

MIDTERM CHECK · TOPIC B

Q6	**0 and 0**	"cat" and "dog" are never adjacent (they never share a document).
Q7	**8/9 ≈ 0.889**	Dot product 8, both lengths 3, so 8 / 9.
Q8	**see B.5**	**Positive:** the target word paired with a word that actually appears in its context window in the training text. **Negative:** the target paired with a randomly sampled noise word from the vocabulary that wasn't observed as its neighbor. The model learns to give positive pairs high $\sigma(\mathbf{w}\cdot\mathbf{c})$ and negative pairs low.
Q9	**D**	Analogy tests with vector arithmetic (king − man + woman ≈ queen). Convergence speed (A) and parameter count (B) say nothing about meaning. Classification (C) is an indirect, extrinsic test, not the common way to check semantic relationships.
Q10	**C**	A pre-trained sentence embedding model (SBERT) is trained so that cosine similarity reflects sentence meaning. Mean or max pooling of word2vec (A, B) loses order and context. One-hot bag-of-words (D) treats "fast" and "quick" as completely unrelated (orthogonal).

TOPIC C · WEEK 4 SLIDES · MIDTERM Q11–Q17

C Neural language models

YOU SHOULD BE ABLE TO

- Define a logit, and compute softmax by hand
- Compute cross-entropy loss and say what makes it large or small
- Explain gradient descent and choose how to change the learning rate
- Describe softmax regression as a next-token predictor
- Explain why stacked linear layers collapse into one
- Count every trainable parameter in a small network

C.1 The neural unit

A **neural unit** takes a weighted sum of its inputs plus a **bias**, then applies a nonlinear **activation function** f:

$$z = \mathbf{w} \cdot \mathbf{x} + b \qquad a = f(z)$$

✎ WORKED EXAMPLE · ONE UNIT (WEEK 4 SLIDES)

$\mathbf{w} = [0.2, 0.3, 0.9]$, $b = 0.5$, $\mathbf{x} = [0.5, 0.6, 0.1]$, sigmoid activation

1. $z = 0.2(0.5) + 0.3(0.6) + 0.9(0.1) + 0.5$
   $= 0.10 + 0.18 + 0.09 + 0.5 = 0.87$
2. $\sigma(0.87) = \dfrac{1}{1 + e^{-0.87}} = \dfrac{1}{1 + 0.419} = 0.70$

Output **y ≈ 0.70**. With ReLU it would be 0.87, and with tanh 0.70.

Activation	Formula	Range
Sigmoid	$\dfrac{1}{1+e^{-z}}$	(0, 1)
tanh	$\dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	(−1, 1)
ReLU	$\max(0, z)$	[0, ∞)

ReLU is the most common. Sigmoid and tanh *saturate*: for large $|z|$ their slope is nearly 0, so the error signal shrinks layer after layer. This is the **vanishing gradient** problem. ReLU's slope is 1 for any $z > 0$.

C.2 Why nonlinear activations matter Q16

A single **perceptron** is a linear classifier: its decision boundary $w_1 x_1 + w_2 x_2 + b = 0$ is a straight line. That works for AND and OR, but **XOR** is **not linearly separable**: no single line separates {01, 10} from {00, 11} (Minsky & Papert, 1969). A **hidden layer** with ReLU solves it:

$h_1 = \text{ReLU}(x_1 + x_2)$, $h_2 = \text{ReLU}(x_1 + x_2 - 1)$, $y = h_1 - 2h_2$

x_1	x_2	h_1	h_2	y
0	0	0	0	**0**
0	1	1	0	**1**
1	0	1	0	**1**
1	1	2	1	**0**

The hidden layer maps both (0,1) and (1,0) to h = (1, 0), which makes the problem linearly separable. Hidden layers **learn useful representations**.

KEY IDEA · LINEAR LAYERS COLLAPSE

Remove the ReLU and the same network computes $y = (x_1 + x_2) - 2(x_1 + x_2 - 1) = -x_1 - x_2 + 2$. That gives 2, 1, 1, 0 for the four inputs, which is a straight line and not XOR. In general:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{(\mathbf{W}_2\mathbf{W}_1)}_{\mathbf{W}'}\mathbf{x} + \underbrace{(\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)}_{\mathbf{b}'}$$

Any stack of linear layers equals one linear layer, so depth adds nothing without nonlinearity.

⚠ EXAM TRAP · Q16

Replacing ReLU with f(x) = x does **not** produce negative probabilities (B), because softmax still makes valid probabilities. Speed (C) is not "the most significant consequence for the ability to learn", and vocabulary size (D) is unrelated. The answer is **(A): the network becomes equivalent to a single linear model.**

C.3 Logits and softmax Q11 Q12

A **logit** is the **raw, unnormalized score** z that the last linear layer produces for each class or token, before softmax. Logits can be any real number, including negative ones. **Softmax** turns a vector of logits into a probability distribution:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

Properties: every output is positive and the outputs sum to 1. The ranking is the same as the logits. No output is ever exactly 0 or 1, so the result is never one-hot. Gaps between logits are exaggerated, because each step of +1 in a logit multiplies its share by $e \approx 2.718$. Adding the same constant to every logit changes nothing. Softmax generalizes sigmoid to k classes.

✎ WORKED EXAMPLE · Q12: SOFTMAX OVER FOUR CANDIDATE TOKENS

Logits after "The cat sat on the": mat = 2.0, floor = 1.0, roof = 0.5, sky = 0.0

1. Exponentiate each logit: $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.5} = 1.649$, $e^{0} = 1.000$
2. Add them: $7.389 + 2.718 + 1.649 + 1.000 = 12.756$
3. Divide each by the sum:
   mat $= 7.389/12.756 =$ **0.579** · floor $= 2.718/12.756 =$ **0.213** · roof $= 1.649/12.756 =$ **0.129** · sky $= 1/12.756 =$ **0.078**
4. Check: $0.579 + 0.213 + 0.129 + 0.078 = 0.999 \approx 1$ (rounding). Every value is positive and mat is highest.

This matches **(D)**. Compare distractor (A), "divide each logit by 3.5": that gives 0.571, 0.286, 0.143 and **0.000**. sky would get zero probability and the ratios would be linear. With softmax, sky still gets 7.8%, and mat/floor = $e^1 \approx 2.7\times$, not 2×.

Temperature (Week 7) divides the logits by τ before softmax. With τ = 0.5 the same logits give mat 0.831, floor 0.112, roof 0.041, sky 0.015, which is *sharper*. With τ = 2 they give 0.409, 0.248, 0.193, 0.150, which is *flatter*. As τ → 0 the result approaches greedy decoding.

C.4 Cross-entropy loss Q13

A **loss function** measures how far the prediction $\hat{\mathbf{y}}$ is from the truth $\mathbf{y}$, where $\mathbf{y}$ is a one-hot vector with 1 for the correct token. **Cross-entropy loss** (also called negative log-likelihood) is:

$$L_{CE} = -\sum_{i} y_i \log \hat{y}_i = -\log \hat{y}_{\text{correct}}$$

✎ WORKED EXAMPLE · Q13: CROSS-ENTROPY

P(mat) = 0.60, P(floor) = 0.25, P(roof) = 0.10, P(sky) = 0.05. The correct token is **mat**.

1. Write the full sum: $-[1 \cdot \log 0.60 + 0 \cdot \log 0.25 + 0 \cdot \log 0.10 + 0 \cdot \log 0.05]$
2. The one-hot zeros remove every term except the correct token's: $L = -\log 0.60$
3. Use the natural log: $\ln 0.60 = -0.511$, so $L = \mathbf{0.511}$. (In base 2 it is 0.737 bits. Either base is fine if used consistently.)
4. If the correct token had been sky: $-\ln 0.05 = 2.996$. **The less probability the model gave the right answer, the larger the loss.**

Answer **(A)**: −log(0.60). Chaining with Q12: P(mat) = 0.579 → loss = −ln 0.579 = 0.546.

P(correct)	Loss −ln p
0.99	0.010
0.90	0.105
0.60	0.511
0.50	0.693
0.25	1.386
0.10	2.303
0.05	2.996
0.01	4.605

The loss is 0 only when p = 1. The **training loss** is the average of −log p over every token position in a batch. A model that guesses uniformly scores ln |V| (for |V| = 100, that is 4.61), so a freshly initialized model starts near this value.

⚠ EXAM TRAP · Q13

The loss is **not** 1 − p (B), **not** an average of all four probabilities (C), and **not** zero just because the correct token got the highest probability (D). A correct guess made with 60% confidence still costs 0.511.

C.5 Gradient descent and the learning rate Q14

Training minimizes the loss. **Backpropagation** computes the **gradient**, which says how the loss changes as each weight changes; PyTorch does this with `loss.backward()`. **Gradient descent** then moves every weight a small step *against* the gradient, scaled by the **learning rate** η:

$$w^{t+1} = w^t - \eta \frac{dL}{dw}$$

Each training step is a forward pass, then the loss, then a backward pass (gradients), then an update. **Stochastic gradient descent (SGD)** uses one example or a small minibatch per step. One **epoch** is one full pass over the training data.

✎ WORKED EXAMPLE · HOW THE LEARNING RATE CHANGES TRAINING

Minimize $L(w) = (w-3)^2$, whose gradient is $dL/dw = 2(w-3)$. Start at $w = 0$; the best value is $w = 3$.

1. First step with η = 0.1: the gradient at 0 is $2(0-3) = -6$, so $w_1 = 0 - 0.1(-6) = 0.6$.

2 Repeat for five steps with three different learning rates:

η	w after steps 0 → 5	Behavior
0.01	`0 → 0.06 → 0.119 → 0.176 → 0.233 → 0.288`	**Too small:** crawls, and the loss falls very slowly
0.1	`0 → 0.6 → 1.08 → 1.464 → 1.771 → 2.017`	**Good:** steady progress toward 3
1.1	`0 → 6.6 → −1.32 → 8.184 → −3.221 → 10.465`	**Too large:** overshoots and diverges

Q14: the loss falls *very slowly*, so the steps are too small. **Increase the learning rate (B).** If the loss jumped around or blew up, you would decrease it instead (C). Removing softmax (A) or enlarging the vocabulary (D) doesn't affect step size.

C.6 Softmax regression as a language model Q15

Softmax regression (multinomial logistic regression) computes $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ and then $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$. Used as a **language model**:

- $\mathbf{x}$ is a fixed feature vector representing the context "The cat sat on the".
- $\mathbf{W}$ has **one row per vocabulary token** (|V| × d), and $\mathbf{b}$ has |V| entries. Row $i \cdot \mathbf{x} + b_i$ is the **linear score (logit) for token *i***.
- Softmax turns all |V| scores into one probability distribution over the next token. At inference you pick the most probable token (greedy) or sample one.

It is a **1-layer network** (the input layer isn't counted) and a **linear** model with no hidden layer. Its parameters are |V|·d + |V|: for d = 512 and |V| = 32,000 that is 16,416,000.

⚠ EXAM TRAP · Q15

The answer is **(B)**. (A) is wrong because softmax regression is one joint softmax over all tokens, not independent binary classifiers. (C) describes recurrent or transformer generation with hidden states. (D) describes an n-gram model, which memorizes counts.

C.7 A feedforward neural language model

Take a **sliding window** of the previous N tokens, say 3. Look up each token's row in the **embedding matrix** E (|V| × d), **concatenate** the rows into $\mathbf{x}$ (length N·d), compute a hidden layer $\mathbf{h} = \text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b})$, then logits $\mathbf{z} = \mathbf{U}\mathbf{h} + \mathbf{b}_2$ and $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$. For example, $\hat{y}_{42}$ is P(next word = V_{42} = "fish").

- **Why this beats n-grams:** suppose training data contained "make sure that the *cat* gets fed" but never "dog gets fed". An n-gram model can't predict "fed" after "the dog gets". A neural LM can, because the *cat* and *dog* embeddings are similar.
- **Two ways to feed embeddings to a classifier:** *concatenate* them (fixed length, so pad or truncate to the context window; 4 tokens × 300 d = 1,200 d) or *pool* them (mean or max, giving one 1 × d vector).
- In code (Week 4): `nn.Linear(context_size*embedding_dim, num_neurons)` → ReLU → `nn.Linear(num_neurons, vocab_size)`.

C.8 Counting parameters Q17

Layer	Trainable parameters	Why
`Embedding(|V|, d)`	**|V| × d**	One d-dimensional row per token. No bias.

Layer	Trainable parameters	Why
`Linear(in → out)`	**in × out + out**	A weight matrix plus **one bias per output neuron**.
ReLU, sigmoid, softmax, flatten/concat	**0**	Fixed functions; nothing is learned.
Context length N	0 by itself	But it sets the first Linear layer's input size: N × d.
One transformer layer (approx.)	**12 d²**	Attention 4d² (W^Q, W^K, W^V, W^O) plus FFN 8d² (d×4d + 4d×d).

✎ WORKED EXAMPLE 1 · A FEEDFORWARD LM (NEW NUMBERS)

|V| = 1,000, embedding d = 16, context N = 3, hidden = 32. Embedding → Flatten → Linear(48 → 32) → ReLU → Linear(32 → 1,000)

1. Embedding: $1{,}000 \times 16 = \mathbf{16{,}000}$
2. Flatten: the input to the next layer is $3 \times 16 = 48$ numbers (0 parameters)
3. Linear(48 → 32): $48 \times 32 + 32 = 1{,}536 + 32 = \mathbf{1{,}568}$
4. ReLU: 0 · Linear(32 → 1,000): $32 \times 1{,}000 + 1{,}000 = \mathbf{33{,}000}$

Total = 16,000 + 1,568 + 33,000 = **50,568**. Notice that 49,000 of these (97%) sit in the embedding and output layers, and both scale with |V|. That is the large-vocabulary cost from Topic A.

✎ WORKED EXAMPLE 2 · Q17: ONELAYERNEURALLM

|V| = 100, d = 8, context = 4, hidden = 4. Embedding → Flatten → Linear(32 → 4) → ReLU → Linear(4 → 100)

1. Embedding: $100 \times 8 = \mathbf{800}$
2. Flatten: $4 \times 8 = 32$ inputs (0 parameters)
3. Linear(32 → 4): $32 \times 4 + 4 = 128 + 4 = \mathbf{132}$
4. ReLU: 0 · Linear(4 → 100): $4 \times 100 + 100 = 400 + 100 = \mathbf{500}$

Total = 800 + 132 + 500 = **1,432 → (C)**. Where the distractors come from: 632 forgets the embedding (132 + 500), 932 forgets the output layer (800 + 132), and 1,632 overcounts by 200.

✎ WORKED EXAMPLE 3 · COUNTING AT SCALE: GPT-3 (WEEK 5 SLIDES)

n = 96 layers, d = 12,288. Count the non-embedding parameters.

1. Attention per layer: four d × d matrices, $4d^2 = 4 \times 12{,}288^2 = 603{,}979{,}776 \approx 604\text{M}$
2. FFN per layer: $d \times 4d + 4d \times d = 8d^2 \approx 1.21\text{B}$. So one layer has $12d^2 \approx 1.81\text{B}$.
3. All layers: $12 \times 96 \times 12{,}288^2 = 173{,}946{,}175{,}488 \approx \mathbf{175}\text{B}$

This ignores biases, LayerNorm, and the embedding matrix (GPT-3's 50,257-token vocabulary × 12,288 ≈ 618M). Scaling laws count the **non-embedding** parameters N.

MIDTERM CHECK · TOPIC C

Q11	**B**	A logit is the raw, unnormalized score before any activation. (A) is the softmax output, (C) is the loss, and (D) is the gradient.
Q12	**D**	Softmax gives all-positive values that sum to 1, with the largest on “mat” (0.579 / 0.213 / 0.129 / 0.078).
Q13	**A**	L = −log(0.60) = 0.511: the penalty depends on the probability given to the correct token.
Q14	**B**	Slow decrease means the steps are too small, so increase the learning rate.
Q15	**B**	A linear score for every token, then softmax over the whole vocabulary.
Q16	**A**	Without nonlinearity the stacked layers collapse into one linear model.
Q17	**C · 1,432**	800 (embedding) + 132 (32×4 + 4) + 500 (4×100 + 100).

TOPIC D · WEEK 5 SLIDES · MIDTERM Q18–Q23

D Transformers & attention

YOU SHOULD BE ABLE TO

- Explain why static embeddings fail for words like “it”
- Describe the roles of query, key and value, and walk through the 6 attention steps
- Compute a small attention output by hand
- Explain why multi-head attention splits dimensions
- Describe the residual stream and why positional embeddings are needed
- Give the LM head’s input and output shapes

D.1 Static vs. contextual embeddings Q19

A **static embedding** such as word2vec or GloVe gives **one fixed vector per word**, whatever the context. Consider:

- “The chicken didn’t cross the road because **it** was too *tired*.” Here it = chicken.
- “The chicken didn’t cross the road because **it** was too *wide*.” Here it = road.

word2vec gives “it” the same vector in both sentences, so it cannot represent the difference.

A **contextual embedding** gives **each occurrence** of a word its own vector, which depends on the surrounding words. Transformers compute contextual embeddings with attention. They are the **final hidden states after the transformer layers**, not the input embeddings. Across the layers, the vector for “it” moves toward “chicken” or toward “road”.

⚠ EXAM TRAP · Q19

The root cause is **(D): one vector per word regardless of context**. More training data (A) can’t help, because the result is still one vector. It is true that word2vec has no positional embeddings or attention (B, C), but that is not *why* it fails here. The failure follows from the definition of a static embedding.

D.2 Attention: a weighted sum of earlier tokens

Attention lets a token “see” earlier tokens. The output for token i is a **weighted sum** of vectors from tokens 1 … i, with each weight set by how similar that token is to token i. In GPT-style models attention runs **left to right**: a **causal mask** sets the scores for future positions to $-\infty$ before softmax, so those positions get weight 0.

✎ WORKED EXAMPLE · SIMPLIFIED ATTENTION (WEEK 5 SLIDES)

$\mathbf{x}_1 = [1, 0, 1]$, $\mathbf{x}_2 = [0, 1, 1]$, $\mathbf{x}_3 = [1, 1, 0]$. Compute the output $\mathbf{a}_3$ for token 3, using the raw vectors as query, key and value.

1. Score token 3 against every token up to and including itself: $\mathbf{x}_3 \cdot \mathbf{x}_1 = 1$, $\mathbf{x}_3 \cdot \mathbf{x}_2 = 1$, $\mathbf{x}_3 \cdot \mathbf{x}_3 = 2$
2. Softmax: $e^1 = 2.718$, $e^1 = 2.718$, $e^2 = 7.389$, sum = 12.826, so $\alpha_{3,1} = 0.212$, $\alpha_{3,2} = 0.212$, $\alpha_{3,3} = 0.576$

3 Weighted sum: $\mathbf{a}_3 = 0.212[1,0,1] + 0.212[0,1,1] + 0.576[1,1,0] = [0.788,\ 0.788,\ 0.424]$

Real attention adds two things. First, separate **query, key and value projections** (D.3). Second, **scaling by $\sqrt{d_k}$**: dividing these scores by √3 gives 0.577, 0.577, 1.155, so α = 0.264, 0.264, 0.471. Scaling softens the distribution.

D.3 Query, key and value Q18

Every token plays three roles, so each input vector $\mathbf{x}$ is projected three ways by learned matrices:

Role	Computed as	What it does	Library analogy
Query q	$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q$	Represents the **current token** being processed. It is compared with every earlier token's key.	Your search request
Key k	$\mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K$	Represents a token as something **being compared against**. It decides how relevant token *j* is.	The catalog card
Value v	$\mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$	The **content that gets passed along**, weighted and summed into the output (a "read-only message").	The book's contents

The six steps in the exam's diagram (output a_3)

1. Generate key, query and value vectors for x_1, x_2 and x_3.
2. **Compare x_3's query with the keys** of x_1, x_2 and x_3 (dot products).
3. Divide each score by $\sqrt{d_k}$.
4. Turn the scores into weights $\alpha_{3,1}$, $\alpha_{3,2}$, $\alpha_{3,3}$ with softmax.
5. Weight each **value** vector by its α.
6. Sum the weighted values to get a_3.

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}_j\big(\text{score}(\mathbf{x}_i, \mathbf{x}_j)\big) \quad \forall j \le i$$

$$\mathbf{a}_i = \sum_{j \le i} \alpha_{ij} \mathbf{v}_j$$

Matrix form: $\text{softmax}\big(\text{mask}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_k})\big)\mathbf{V}$, where $\mathbf{Q}\mathbf{K}^\top$ is N × N. **Why $\sqrt{\mathbf{d_k}}$?** Dot products grow with dimension, and huge scores make softmax nearly one-hot, which causes numerical problems and vanishing gradients.

⚠ EXAM TRAP · Q18

The answer is **(C)**: the query is compared with every token's key to produce the attention scores. It is never added to the values (A). Softmax is applied to the query·key *scores*, not to the query itself (B). Nothing gets replaced (D).

D.4 Multi-head attention Q20

Multi-head attention runs *h* attention **heads** in parallel. Each head *c* has its own $\mathbf{W}_c^Q, \mathbf{W}_c^K, \mathbf{W}_c^V$, which project to **d/h** dimensions. The head outputs are **concatenated** back to d and multiplied by $\mathbf{W}^O$ (d × d).

EXAMPLE FROM THE SLIDES

A 1024-dimensional embedding split into **4 heads of 256**. Each head attends separately and outputs 256 numbers. 4 × 256 is concatenated to 1024, then multiplied by W^O. Llama 3 405B uses up to 128 heads.

KEY IDEA · WHY SPLIT?

So different heads can attend to different kinds of relationships at the same time. One head might link "it" to "chicken" (coreference), another might track the previous word, another subject–verb agreement. Splitting adds no parameters: the heads together still have d × d for each of Q, K and V.

⚠ EXAM TRAP · Q20

Splitting into heads doesn't change the vocabulary (A). The $\sqrt{d_k}$ scaling, not splitting, is what stabilizes softmax (B). Each head outputs 256 dimensions, *fewer* than the input's 1024, not the same (D). **Answer: (C).**

D.5 The transformer block and the residual stream Q23

One **transformer block** (pre-norm), for token i:

$$\begin{aligned}
\mathbf{t}_i^1 &= \text{LayerNorm}(\mathbf{x}_i) \\
\mathbf{t}_i^2 &= \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \ldots, \mathbf{t}_N^1]) \\
\mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \quad \leftarrow \text{add} \\
\mathbf{t}_i^4 &= \text{LayerNorm}(\mathbf{t}_i^3) \\
\mathbf{t}_i^5 &= \text{FFN}(\mathbf{t}_i^4) \\
\mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3 \quad \leftarrow \text{add}
\end{aligned}$$

$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$, going d → 4d → d. **Layer norm** z-scores a *single token's* vector (subtract its mean, divide by its standard deviation), then applies a learned scale and shift.

KEY IDEA · THE RESIDUAL STREAM

Each token has **one d-dimensional vector that flows upward through every block**. This is its **residual stream**. Each component (attention, FFN) **reads from the stream and adds its output back into it**. Nothing ever replaces the stream. Every vector stays d-dimensional, which is why blocks can be stacked.

Attention is the only component that moves information between token positions. It copies information from neighboring tokens' streams into the current one (Elhage et al., 2021). The FFN and LayerNorm act on one token at a time.

⚠ EXAM TRAP · Q23

Answers that say "replaces" (A, B) are wrong; the + residual connection exists precisely to avoid replacement. (D) is wrong because the feedforward network also adds to the stream. **Answer: (C).**

D.6 Input: token + positional embeddings Q22

Each input vector is the **sum of two embeddings**: $\mathbf{x}_i = \mathbf{E}[w_i] + \mathbf{E}_{pos}[i]$. The **token embedding** is row w_i of E (|V| × d). The **positional embedding** is row i of E_{pos}, a learned d-dimensional vector for each position up to the maximum length. Example: "Thanks for all the" → IDs [5, 4000, 10532, 2224] → rows 5, 4000, 10532 and 2224 of E, plus the vectors for positions 1–4, giving X (4 × d).

Why positions are needed: self-attention computes weighted sums over a *set* of tokens, and nothing in it depends on order. "The dog bit the man" and "The man bit the dog" contain exactly the same tokens, so without positional embeddings the transformer would treat them as identical and couldn't tell who bit whom. **Q22 → (B).** Attention doesn't track order by itself (A). Unique token embeddings say *which* words are present, not *where* they are (D). Learned absolute positions are the simplest scheme; other schemes such as sinusoidal and rotary also exist.

D.7 Output: the language modeling head Q21

The **language modeling head** takes the **final-layer hidden state of the last token**, $\mathbf{h}_N^L$ (1 × d). It multiplies by the **unembedding** matrix E^T (d × |V|), which takes the dot product with every token's embedding. That gives **logits** (1 × |V|), and softmax turns them into probabilities over the vocabulary.

Q21: with d = 512 and |V| = 32,000, the input is the 512-dim hidden state of the last token and the output is **32,000 logits, one per vocabulary token → (D)**.

Training vs. generation (Week 6): during training the LM head runs at *every* position, so each position predicts its next token and all the predictions are checked at once. During generation only the last position is used; the chosen token is appended and the model runs again. The **KV cache** stores earlier tokens' keys and values so they aren't recomputed at each step.

Shape tracker	Shape	d = 512, |V| = 32K, N = 5
Token IDs	N	5
X = E[w] + E_{pos}	N × d	5 × 512
Each transformer block	N × d → N × d	5 × 512
Q, K, V for one head (8 heads)	N × d_k	5 × 64
QK^T (attention scores)	N × N	5 × 5
LM head input (last token)	1 × d	1 × 512
Unembedding E^T	d × |V|	512 × 32,000
Logits → probabilities	1 × |V|	1 × 32,000

MIDTERM CHECK · TOPIC D

Q18	C	x_3's query is compared (dot product) with every token's key. The scores, after $\sqrt{d_k}$ and softmax, set how much each value contributes to a_3.
Q19	D	Static embeddings give a word the same vector regardless of context.
Q20	C	Different heads can attend to different kinds of relationships at the same time.
Q21	D	Input: the 512-dim hidden state of the last token. Output: 32,000 logits, one per vocabulary token.
Q22	B	Self-attention has no built-in notion of order, so the two sentences would look identical.
Q23	C	Each component reads from the residual stream and adds its output back into the same stream.

TOPIC E · WEEKS 5–7 SLIDES · MIDTERM Q24–Q30

E Shaping model behavior

YOU SHOULD BE ABLE TO

- Name the training phases and what each one teaches
- Compare SFT data with preference data
- Define the policy, reward model and KL penalty in RLHF
- Recognize in-context (few-shot) learning
- Read a loss table and choose the early-stopping epoch
- State the three factors in scaling laws

E.1 Two levers: change the weights, or change the context

	Developers change the weights (training)	Users change the context (prompting)
When	Before release, during training	Every time a message is sent
How	Pretraining, SFT, RLHF/DPO, RLVR	System prompt, examples, RAG, settings (temperature)
Lasts	Permanently, for every user	For this conversation only. A chatbot's weights are frozen.

E.2 The training phases Q24

Phase	Data	Method	Learns
1. Pretraining	Trillions of tokens of web text, code and books	**Self-supervised** next-token prediction (cross-entropy at every position)	Language and world knowledge. The result is a **base model**.
2. SFT (supervised fine-tuning, or instruction tuning)	Prompt → answer pairs written by people or stronger models	The *same* next-token loss, counted **only on the answer tokens**	Following instructions and chatting
3. Preference alignment (**RLHF** or DPO)	A prompt plus a **chosen** and a **rejected** answer	Reward model + PPO (reinforcement learning), or DPO	A helpful, safe tone and style
4. RLVR (modern)	Math, code and tool tasks with checkable answers	GRPO: reward = 1 only if a verifier says the answer is correct	Multi-step reasoning

For Q24, the classic three are pretraining → SFT → RLHF. Week 6 adds RLVR as a fourth modern stage. Labs repeat stages 2–4 in several rounds, and DeepSeek-R1-Zero skipped SFT entirely: RLVR alone took its AIME accuracy from 15.6% to 71.0%.

Pretraining details

- Training predicts the next token at **every position at once**, because each position sees only earlier tokens.
- It can run in stages. Qwen3 did general pretraining (~30T tokens), then mid-training on STEM, code and reasoning (~5T), then long-context training (<1T, up to 32K tokens).
- Data curation matters as much as quantity: filter, deduplicate, extract text from PDFs, synthesize data, and tune the mix.
- **The data wall:** there are roughly 300T tokens of quality public text, which training could exhaust between 2026 and 2032. Labs respond with synthetic data, other modalities, repeated data, and more post-training.

SFT details

- Each example is formatted as a chat with system, user and assistant roles (e.g. ChatML `<|im_start|>user … <|im_end|>`). Using the wrong template at inference degrades the output.
- The loss is **ignored on the prompt** and counted on the answer.
- It uses far less data (thousands to millions of examples), runs for a **few passes at a small learning rate** so the model keeps what it knows, and can update all weights or just small add-on **LoRA** matrices.
- **Evaluation:** exact match fails, because "as fast as a bullet" is a valid answer even when the reference says "lightning". Chat models are judged by benchmarks (MMLU-Pro, GPQA; these saturate), human votes (Arena), and an **LLM-as-judge** scoring against a rubric (which can favor long answers).

E.3 SFT data vs. preference data Q25

	SFT data (demonstrations)	Preference data (comparisons)
One example has	A prompt plus **one complete, correct target response**	A prompt plus **two or more responses**, and a label saying which one people preferred (**chosen** vs. **rejected**). There is no gold answer.
Example	"Identify the correct spelling: Ocassion" → "The correct spelling is 'Occasion'."	"Does garlic help with colds?" ✓ "It can be helpful against colds, but may make you stink." × "It might be one of the best natural antibiotics out there..."
Who makes it	Annotators, or a stronger model, *write* the answer	Annotators, online votes or LLM judges *pick* the better answer. InstructGPT raters ranked 4 answers per prompt, giving 6 pairs.
Analogy	Copying worked examples from a teacher	A boss who reads two drafts and says which is better
Why use it	Shows the format and the desired behavior	**Easier to give:** judging needs only an opinion, not the skill to write the best answer

⚠ EXAM TRAP · Q25

Both kinds of data contain the prompt (B). Multiple candidate responses (C) are in *preference* data, not SFT data. A numerical reward (D) comes from the RLHF side. Only SFT has **(A): a complete, correct target response written by a human**.

E.4 The reward model, the RL vocabulary, and the policy Q30

A **reward model** is the SFT model with its LM head (|V| scores) **replaced by a head that outputs one number**, read at the last position. It is trained on preference pairs so that the chosen answer scores higher than the rejected one, using the **Bradley-Terry** model:

$$P(\text{chosen} \succ \text{rejected}) = \sigma(r_c - r_r)$$

$$L = -\log \sigma(r_c - r_r)$$

✎ WORKED EXAMPLE · BRADLEY–TERRY (WEEK 6 NUMBERS)

1. **Before training:** chosen 0.3, rejected 0.5. Gap −0.2, so σ(−0.2) = 0.450 and loss = −ln 0.450 = **0.80**. The order is wrong, so the loss is high.
2. **After training:** chosen 2.1, rejected −0.8. Gap 2.9, so σ(2.9) = **0.948 ≈ 95%** and loss = 0.054.

A gap of 0 means 50/50. This is just cross-entropy (Topic C) applied to "which answer wins?".

RL terms, applied to an LLM	
Policy	**The LLM being trained.** Its next-token probabilities are its strategy, and they are updated to maximize reward.
State	The prompt plus the tokens written so far
Action	Choosing the next token
Reward	The reward model's score for the finished answer

The RLHF loop: pick a prompt, the policy writes an answer, the reward model scores it, and **PPO** updates the policy so high-scoring answers become more likely. Repeat.

- **PPO** (Proximal Policy Optimization) uses a value model to predict the usual score. An answer scoring 1.7 against an expected 1.0 is +0.7 better, so it becomes likelier. Steps are capped and kept small. PPO needs **4 models in memory**: the policy, a frozen reference copy, the reward model and the value model.
- **DPO** has no reward model and no RL. It raises the probability of the chosen answer and lowers the rejected one directly, while staying close to the reference. It needs 2 models; Llama 3 used it.
- **GRPO** (used for RLVR) samples a group of attempts, e.g. 8 tries with 3 correct, average 0.375. Each try is judged against the group average: correct tries get +0.625 and wrong ones −0.375. No value model is needed.

⚠ EXAM TRAP · Q30

The policy is **(C): the language model itself**. The preference comparisons (A) are the reward model's training data. The rule for how much parameters change (B) is the optimizer and learning rate. The scoring function (D) is the reward model.

E.5 The KL penalty Q29

The problem: the reward model is only an *estimate* of human preference. A policy that maximizes reward without limits learns to exploit the reward model's blind spots (**reward hacking**), for example with overly long, flattering or repetitive answers that happen to score well. It can also drift far from fluent language and **forget what pretraining and SFT taught it**.

The fix: keep a frozen copy of the starting (SFT) model as a **reference model** and subtract a penalty for diverging from it. The **KL divergence** measures how different two probability distributions are, and it is 0 when they are identical:

$$R(x, y) = r_{RM}(x, y) - \beta \, \mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{ref}(\cdot \mid x)\big)$$

For a single sampled answer, the KL term is estimated as $\log \pi_\theta(y \mid x) - \log \pi_{ref}(y \mid x)$. The coefficient β sets how strongly the model is kept near its start. The slides put it this way: "the reward is only an estimate, so training keeps the model close to where it started."

✎ WORKED EXAMPLE · THE PENALTY AT WORK (ILLUSTRATIVE NUMBERS)

β = 0.1. The reward model gives two answers the same score, 1.7.

1. **Answer A** (a reward-hacking answer): the policy now gives it log-prob −20, while the reference model gave it −35. Log ratio = −20 − (−35) = 15.
2. Penalty = 0.1 × 15 = 1.5, so the net reward is 1.7 − 1.5 = **0.2**.
3. **Answer B** (one the original model also found plausible): log ratio = 1, so the penalty is 0.1 and the net reward is **1.6**.

The penalty keeps the policy close to the pretrained/SFT distribution. RL refines behavior instead of drifting into whatever the reward model happens to overrate.

BEYOND THE SLIDES

The explicit KL formula comes from the RLHF literature (Ziegler et al., 2019; InstructGPT, 2022). The slides express the same idea through PPO's "frozen copy" and DPO's reference model.

⚠ EXAM TRAP · Q29

The KL penalty doesn't limit the model to high-scoring responses only (B). It *lowers* rewards rather than raising them (C). It has nothing to do with training speed (D). **Answer: (A), to stop the model diverging too far from the pretrained distribution and forgetting prior knowledge.**

E.6 In-context (few-shot) learning Q26

In-context learning is learning that improves the model's performance from **instructions or examples placed in the prompt, with no parameter updates**. Only the context, and therefore the network's activations, changes (the term comes from the GPT-3 paper, Brown et al., 2020). A prompt can be **zero-shot** (instruction only), **one-shot** (one demonstration) or **few-shot** (several demonstrations).

✎ Q26 · NAME THE TECHNIQUE

```
The battery life is amazing. → positive
The screen cracked after one day. → negative
Shipping was fast and the product works great.
→ positive
[new review here]
```

Few-shot prompting, i.e. in-context learning: three demonstrations, no fine-tuning, and the weights are untouched.

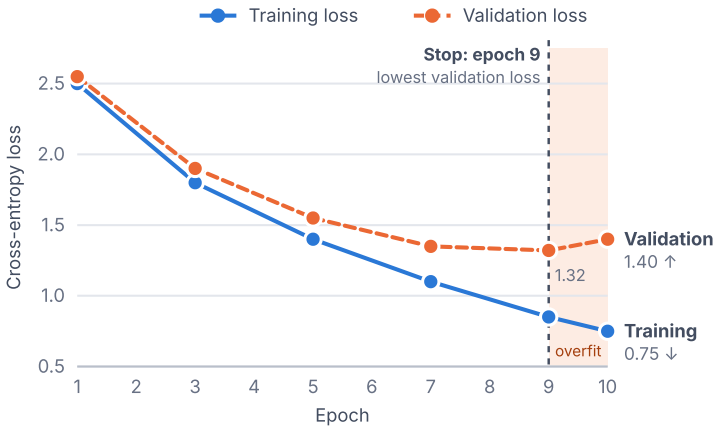
- **Tasks as conditional generation:** for sentiment, compare P("positive" | "The sentiment of the sentence 'I like Jackie Chan' is:") with P("negative" | …).
- **System prompt**: instructions silently prepended to every conversation. Claude's runs to about 1,700 words. API developers write their own. A single added verbosity instruction (April 2026) hurt coding quality and was reverted.
- **Five techniques:** role, examples (few-shot), format, step-by-step, and grounding context.
- **Prompts are fragile:** reordering the same few-shot examples moved accuracy from near chance to near state of the art, and formatting changes alone moved it by up to 76 points. **Prompt injection**: text inside a web page ("Ignore your instructions…") can hijack an assistant, because all text shares one context.

E.7 Overfitting and early stopping Q27

Training loss is measured on the data the model learns from. **Validation loss** is measured on **held-out data it never trains on**, so it estimates how well the model generalizes. **Overfitting** is when training loss keeps falling while validation loss stops improving and starts to rise: the model is memorizing details of its training data instead of general patterns. (**Underfitting**: both losses stay high.) **Early stopping** means watching validation loss and **keeping the checkpoint where it is lowest**.

✎ WORKED EXAMPLE · Q27: WHEN SHOULD TRAINING STOP?

Epoch	Training	Validation	Gap
1	2.50	2.55	0.05
3	1.80	1.90	0.10
5	1.40	1.55	0.15
7	1.10	1.35	0.25
9	0.85	**1.32**	0.47
10	0.75	1.40 ↑	0.65



Training loss keeps falling, but validation loss bottoms out at epoch 9 (1.32) and then rises. The shaded area is where the model overfits.

1. Find the minimum of the **validation** column: **1.32 at epoch 9**.
2. Check what happens next: at epoch 10, validation **rises** (1.32 → 1.40) while training keeps **falling** (0.85 → 0.75). That is the signature of overfitting.

3. Stop at, or roll back to, epoch 9. Don't stop at 7: the gap is widening there, but validation loss is *still improving*.

Answer: **epoch 9**, because validation loss is lowest there (1.32). After it, validation loss rises (1.40) even though training loss keeps dropping, which shows the model starting to overfit.

Other remedies for overfitting: more or more varied data, regularization (dropout, weight decay), or a smaller model. SFT uses "a few passes at a small learning rate" for the same reason.

E.8 Scaling laws Q28

Scaling laws (Kaplan et al., 2020): a model's loss improves predictably, as a **power law**, with each of three factors, as long as the other two aren't the bottleneck:

- **N, model size:** parameters, *not counting embeddings*
- **D, dataset size:** training tokens
- **C, compute:** training FLOPs, $C \approx 6ND$

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} \quad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$

Each 10× increase buys about the same improvement, and all three must grow together. Labs use small runs to **predict the loss of a big run** before paying for it.

✎ WORKED EXAMPLE · TOKENS PER PARAMETER

1. **GPT-3:** 175B parameters trained on 300B tokens, about 1.7 tokens per parameter. Compute ≈ $6 \times 175{\times}10^9 \times 300{\times}10^9$ = **3.15×10^{23} FLOPs**. The model was too big for its data.
2. **Chinchilla (2022):** about **20 tokens per parameter** is compute-optimal. A 70B model should see $20 \times 70\text{B} = 1.4\text{T}$ tokens, which takes ≈ $6 \times 70{\times}10^9 \times 1.4{\times}10^{12} \approx 5.9 \times 10^{23}$ FLOPs.
3. **Today:** smaller models trained far longer (open models average about 300 tokens per parameter), because small models are cheaper to serve to millions of users.

⚠ EXAM TRAP · Q28

Vocabulary size, embedding dimension, learning rate, head count, dropout, batch size, optimizer and activation function (A, B, D) are hyperparameters and architecture details. The scaling-law variables are **(C): model size N, dataset size D and compute budget C**.

E.9 Also from Week 7 (outside the exam's key concepts, but fair game)

Concept	Core idea
RAG	**Retrieve** the passages whose embeddings are closest to the question, **augment** the prompt with them, and **generate** an answer that can cite them. It fixes the knowledge cutoff and private data without retraining, but it is only as good as what it retrieves ("lost in the middle"). Parametric knowledge (in the weights) and contextual knowledge (in the prompt) can conflict. Remedies work at the prompt level ("according to Bob..."), decoding level (context-aware decoding), training level (swapped answers, DisentQA) and retrieval level (Self-RAG, Corrective RAG).

Concept	Core idea
Decoding / sampling	**Greedy** decoding always takes the most probable token, which gives generic, repetitive text. **Random sampling** sometimes picks odd tokens from the long tail. **Temperature**: softmax(u/τ), where low τ sharpens and high τ flattens. **Top-k**: keep the k most likely tokens and renormalize (with k = 2, "all" 44% and "the" 33% become 57% / 43%). **Top-p**: keep the smallest set of tokens whose probabilities add up to p, which adapts to how confident the model is.
Inference-time scaling	Spend more compute per question with the same weights: ask for step-by-step reasoning, use **self-consistency** (sample 5 answers at τ = 0.7, get 25, 35, 25, 20, 25, and take the majority, 25), or raise a reasoning-effort budget.
Steering vector	Compute $v = \text{mean}(h_{love}) - \text{mean}(h_{hate})$ at one layer, then add $\alpha \cdot v$ to the hidden states during generation. There is no prompt change and no weight change, but it needs open weights, and a large α breaks fluency. The **logit lens** reads intermediate layers through the LM head.
Training explains behavior	Pretraining explains the knowledge cutoff and confident mistakes. SFT explains the chat format and refusals. Preference alignment explains agreeing too easily (**sycophancy**) and long, polished answers. RLVR explains thinking before answering.

MIDTERM CHECK · TOPIC E

Q24	**3 phases**	**1. Pretraining** (self-supervised next-token prediction) · **2. Supervised fine-tuning** (instruction tuning) · **3. RLHF** (preference alignment). Optionally mention RLVR as a modern fourth stage.
Q25	**A**	SFT data contains a complete, correct target response. Preference data has only comparisons.
Q26	**few-shot**	Few-shot prompting, i.e. in-context learning: examples in the prompt, no weight updates.
Q27	**epoch 9**	Validation loss is lowest at epoch 9 (1.32). At epoch 10 it rises to 1.40 while training loss keeps falling, which signals overfitting, so stop early at 9.
Q28	**C**	Model size N, dataset size D, compute budget C.
Q29	**A**	The KL penalty keeps the policy close to the pretrained/SFT distribution, preventing reward hacking and the loss of prior knowledge.
Q30	**C**	The policy is the language model itself, which is updated to maximize reward.

LAST-MINUTE REVIEW · FORMULA SHEET & ANSWER KEY

Every formula on one page

A · TOKENIZATION

Types = distinct words; instances = every occurrence. |V| = vocabulary size.

BPE: repeat k times: merge the most frequent adjacent pair. The vocabulary grows by 1 per merge.

UTF-8 bytes: ASCII 1 · most European, Middle Eastern, African 2 · CJK 3 · emoji 4.

GPT-4o numbers: digits in chunks of up to 3, left to right (13078 → 130|78). The space before a number is its own token (220).

B · EMBEDDINGS

$$\mathbf{v}\cdot\mathbf{w}=\sum_i v_i w_i \qquad |\mathbf{v}|=\sqrt{\sum_i v_i^2}$$

$$\cos(\mathbf{v},\mathbf{w})=\frac{\mathbf{v}\cdot\mathbf{w}}{|\mathbf{v}||\mathbf{w}|}\in[-1,1]\ \ \text{(counts: } [0,1])$$

Co-occurrence: cell (w, c) = number of times c is within ±window of w. Symmetric. Never cross document boundaries.

$$P(+\mid w,c)=\sigma(\mathbf{c}\cdot\mathbf{w})$$

$$L_{SGNS}=-\left[\log\sigma(\mathbf{c}_{pos}\cdot\mathbf{w})+\sum_{i=1}^{k}\log\sigma(-\mathbf{c}_{neg_i}\cdot\mathbf{w})\right]$$

Analogy: king − man + woman ≈ queen (nearest by cosine).

C · NEURAL LANGUAGE MODELS

$$z=\mathbf{w}\cdot\mathbf{x}+b \qquad \sigma(z)=\frac{1}{1+e^{-z}} \qquad \mathrm{ReLU}(z)=\max(0,z)$$

$$\mathrm{softmax}(z_i)=\frac{e^{z_i}}{\sum_j e^{z_j}} \qquad \text{with temperature: } \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$$

$$L_{CE}=-\sum_i y_i\log\hat{y}_i=-\log\hat{y}_{\text{correct}}$$

$$w^{t+1}=w^t-\eta\,\frac{dL}{dw}$$

Softmax regression LM: $\hat{\mathbf{y}}=\mathrm{softmax}(\mathbf{W}\mathbf{x}+\mathbf{b})$, with W of size |V| × d.

Parameters: Embedding = |V|·d · Linear(in→out) = in·out + out · activations = 0 · transformer layer ≈ $12d^2$.

Linear collapse: $W_2(W_1x + b_1) + b_2 = W'x + b'$.

D · TRANSFORMERS

$$\mathbf{q}_i=\mathbf{x}_i\mathbf{W}^Q \quad \mathbf{k}_j=\mathbf{x}_j\mathbf{W}^K \quad \mathbf{v}_j=\mathbf{x}_j\mathbf{W}^V$$

$$\alpha_{ij}=\mathrm{softmax}_j\left(\frac{\mathbf{q}_i\cdot\mathbf{k}_j}{\sqrt{d_k}}\right),\ j\le i \qquad \mathbf{a}_i=\sum_{j\le i}\alpha_{ij}\mathbf{v}_j$$

Multi-head: h heads of size d/h → concatenate → × W^O.

Block: h = x + Attn(LN(x)), then h = h + FFN(LN(h)). FFN: d → 4d → d.

Input: $x_i = E[w_i] + E_{pos}[i]$. **LM head:** $u = h_N E^T$ (1 × |V|), y = softmax(u).

E · SHAPING BEHAVIOR

Stages: pretraining → SFT → RLHF (preference) → (RLVR).

$$P(c\succ r)=\sigma(r_c-r_r)$$

$$R=r_{RM}(x,y)-\beta\left[\log\pi_\theta(y|x)-\log\pi_{ref}(y|x)\right]$$

RL: policy = the LLM · state = prompt + tokens so far · action = next token · reward = reward model score.

GRPO advantage = reward − group mean. **Early stopping** = the epoch with minimum validation loss.

Scaling: loss is a power law in N, D, C; C ≈ 6ND; Chinchilla ≈ 20 tokens per parameter.

HANDY VALUES FOR HAND CALCULATION

$e^{0.5}$ = 1.649	e^{1} = 2.718	e^{2} = 7.389
e^{3} = 20.09	e^{-1} = 0.368	$e^{-0.87}$ = 0.419
ln 0.9 = −0.105	ln 0.6 = −0.511	ln 0.5 = −0.693
ln 0.25 = −1.386	ln 0.1 = −2.303	ln 0.05 = −2.996
√2 = 1.414	√3 = 1.732	√9 = 3

Spring 2026 midterm: answer key at a glance

Q1 C	**Q2** A	**Q3** B	**Q4** 9300, 4388	**Q5** B
Q6 0 and 0	**Q7** 8/9 ≈ 0.889	**Q8** real neighbor vs. random noise word	**Q9** D	**Q10** C
Q11 B	**Q12** D	**Q13** A	**Q14** B	**Q15** B
Q16 A	**Q17** C (1,432)	**Q18** C	**Q19** D	**Q20** C
Q21 D	**Q22** B	**Q23** C	**Q24** pretrain · SFT · RLHF	**Q25** A
Q26 few-shot / in-context	**Q27** epoch 9	**Q28** C	**Q29** A	**Q30** C

GLOSSARY · EVERY HIGHLIGHTED TERM

Glossary

The colored letter after each term shows the topic where it is explained.

A

Activation function C Nonlinear function applied to a unit's weighted sum z (sigmoid, tanh, ReLU). Without it, stacked layers collapse into one linear layer.

Allocational harm B Harm that occurs when a system unfairly allocates resources such as jobs or credit, e.g. biased embeddings downranking women's résumés.

Analogy test B Intrinsic evaluation of embeddings by vector arithmetic: king − man + woman should be nearest (by cosine) to queen.

Attention D Mechanism that builds a token's new representation as a weighted sum of earlier tokens' value vectors, with weights from query·key similarity.

B

Backpropagation C Algorithm that computes the gradient of the loss with respect to every weight by working backward through the network (loss.backward()).

Base model E The model after pretraining only: it continues text but doesn't reliably follow instructions.

Bias (term) C The constant b added to a unit's weighted sum. A Linear layer has one bias per output neuron.

Bradley–Terry model E Turns two reward scores into a preference probability: P(chosen beats rejected) = $\sigma(r_c - r_r)$. Used to train reward models.

Byte-level BPE A BPE run on UTF-8 bytes instead of characters. All 256 bytes are in the base vocabulary, so any text can be encoded and there is no OOV.

Byte-Pair Encoding (BPE) A Subword algorithm that starts from single symbols and repeatedly merges the most frequent adjacent pair into a new token, k times.

C

Causal mask D Sets the attention scores for future positions to $-\infty$ before softmax, so each token attends only to itself and earlier tokens.

Co-occurrence matrix B $|V| \times |V|$ table whose cell (w, c) counts how often context word c appears within the window around word w. Sparse and symmetric.

Code point A Unicode's unique ID for a character, written like U+0061 ("a"). It is abstract: glyphs live in fonts.

Code switching A Using more than one language in the same utterance, common in speech, texting and social media.

Context window B The span counted as "near" a word when building co-occurrence counts or word2vec pairs (±1, ±4, or a whole document). For an LLM, also the maximum number of tokens it can read at once.

Contextual embedding D A vector for one occurrence of a word that depends on its context: the transformer's final hidden state, e.g. from BERT or GPT.

Cosine similarity B Dot product divided by both vector lengths, which is the cosine of the angle between them. 1 = same direction, 0 = unrelated. It ignores vector length (frequency).

Cross-entropy loss C $-\Sigma\, y_i \log \hat{y}_i$, which equals −log(probability of the correct token). 0 only when that probability is 1. The standard LM training loss.

D

Distributional hypothesis B Words that occur in similar contexts tend to have similar meanings (Harris 1954; Firth 1957).

Dot product B $\Sigma\, v_i w_i$. High when two vectors are large in the same dimensions, but biased toward long (frequent-word) vectors.

DPO E Direct Preference Optimization: trains directly on chosen/rejected pairs with no reward model and no RL, while staying close to a reference model.

E

Early stopping E Halting training (or keeping the checkpoint) at the epoch with the lowest validation loss, before overfitting sets in.

Embedding matrix (E) C $|V| \times d$ lookup table with one learned row vector per token. A token ID selects its row.

Epoch C One full pass through the training data.

F

Few-shot E A prompt that includes several input → output demonstrations before the new input (one-shot = one demonstration).

G

Gradient C Vector of partial derivatives of the loss with respect to the weights. It points uphill, so training steps the opposite way.

Gradient descent C Repeatedly updating $w \leftarrow w - \eta \cdot dL/dw$ to reduce the loss.

GRPO E Group Relative Policy Optimization: samples a group of answers and rewards each one relative to the group's average. No value model is needed. Used in RLVR.

H

Head (attention head) D One of the parallel attention computations in multi-head attention, each with its own W^Q, W^K, W^V of size d × d/h.

Hidden layer C A layer between input and output that learns intermediate representations (e.g. the h layer that makes XOR separable).

I

In-context learning E Better performance from instructions or examples in the prompt, with no parameter updates. Only the context and activations change.

Instance (word instance) A One occurrence of a word in running text. Compare with type.

K

Key D $k_j = x_j W^K$: a token's representation as something being compared against. Its dot product with the current query sets the attention score.

KL divergence E A measure of how much one probability distribution differs from another. 0 when they are identical.

KL penalty E A term $-\beta \cdot KL(\pi_\theta \parallel \pi_{ref})$ subtracted from the RLHF reward. It keeps the policy close to the reference model, preventing reward hacking and forgetting.

KV cache D At inference, stores earlier tokens' key and value vectors so they don't have to be recomputed for each new token.

L

Language model C A model that assigns probabilities to the next token given the previous tokens.

Language modeling head (LM head) D Final layer: takes the last token's final hidden state (1 × d), multiplies by the unembedding E^T (d × |V|) to get one logit per token, then applies softmax.

Layer norm D Z-score normalization of a single token's vector (subtract its mean, divide by its standard deviation), followed by a learned scale and shift.

Learning rate (η) C Step size in gradient descent. Too small means slow training; too large means overshooting or divergence.

LLM-as-judge E Using a strong model with a rubric to score another model's answers. Can be biased toward length or its own style.

Logit C Raw, unnormalized score for a class or token, produced before softmax. Can be any real number.

LoRA E Low-Rank Adaptation: fine-tuning small added matrices instead of all weights, which saves memory.

Loss function C A number measuring how wrong the model's prediction is. Training minimizes it.

M

Multi-head attention D Several attention heads run in parallel on d/h-dimensional projections, and their outputs are concatenated and multiplied by W^O. Different heads can capture different relationships.

N

Negative example (SGNS) B A pair of the target word and a randomly sampled noise word (sampled by weighted unigram frequency) that didn't occur as its neighbor. k per positive.

Neural unit C Computes z = w·x + b, then outputs a = f(z).

O

Out-of-vocabulary (OOV) A A word not in the tokenizer's vocabulary. Word-level tokenizers map it to [UNK]; subword tokenizers build it from known pieces.

Overfitting E Training loss keeps falling while validation loss rises: the model memorizes its training data instead of generalizing.

P

Perceptron C A single unit with a step-function output. It is a linear classifier, so it can learn AND and OR but not XOR.

Policy E In RL for LLMs, the language model being trained: its next-token probabilities, updated to maximize reward.

Pooling B Combining many token vectors into one fixed-size vector: mean pooling averages each dimension, max pooling takes the maximum.

Positional embedding D A learned vector for each position that is added to the token embedding, so the transformer can tell word order.

Positive example (SGNS) B A pair of the target word and a word that actually occurs inside its context window in the corpus.

PPO E Proximal Policy Optimization: the standard RL algorithm in RLHF. It takes small, capped policy updates, using a value model as a baseline.

Pre-tokenization A Regex splitting of text (at spaces, punctuation, digit chunks) before BPE, so merges never cross those boundaries.

Preference alignment E Post-training stage that learns from comparisons of answers (RLHF or DPO), improving helpfulness, safety and style.

Preference data E Prompt + chosen response + rejected response (or a ranking). It contains comparisons, not a gold answer.

Pretraining E Self-supervised next-token prediction on trillions of tokens. Learns language and world knowledge.

Prompt injection E Text inside the model's input (e.g. a web page) that tries to override the user's or developer's instructions.

Q

Query D $q_i = x_i W^Q$: the current token's representation as the one doing the comparing. It is dot-producted with every key.

R

RAG E Retrieval-Augmented Generation: retrieve relevant passages, add them to the prompt, and generate an answer grounded in them.

Reference model E A frozen copy of the starting (SFT) model, used by the KL penalty and DPO to keep the policy close to where it began.

ReLU C Rectified Linear Unit, max(0, z). The most common activation; its gradient is 1 for $z > 0$, which avoids vanishing gradients.

Residual stream D Each token's single d-dimensional vector that flows up through the blocks. Attention and the FFN read from it and add their outputs back into it.

Reward hacking E A policy exploiting the reward model's flaws (e.g. verbosity, flattery) to get high reward without actually being better.

Reward model E The SFT model with its LM head replaced by a single-number head, trained on preference pairs to score answers.

RLHF E Reinforcement Learning from Human Feedback: train a reward model on human preferences, then optimize the policy with RL (PPO) plus a KL penalty.

RLVR E RL with Verifiable Rewards: the reward is an automatic check (correct math answer, passing tests). Teaches multi-step reasoning.

S

Scaling laws E Loss falls as a power law in model size N (non-embedding parameters), data D (tokens) and compute C (≈ 6ND).

Self-supervised learning B Training where the labels come from the data itself, e.g. the next token or a real neighboring word.

Sentence embedding B One dense vector for a whole sentence or document (doc2vec, SBERT). Best suited for clustering and semantic search.

Sentence-BERT (SBERT) B BERT + mean pooling, fine-tuned in a Siamese network on NLI sentence pairs so that cosine similarity reflects sentence meaning.

SFT (supervised fine-tuning) E Continued next-token training on prompt → answer pairs (loss only on the answer). Teaches the model to follow instructions.

SFT data E Instruction or prompt paired with one complete, correct target response written by a person or a stronger model.

SGD (stochastic gradient descent) C Gradient descent that uses one example or a small minibatch per update.

Sigmoid C $\sigma(z) = 1/(1 + e^{-z})$, mapping to (0, 1). Used for binary probabilities. Saturates for large $|z|$.

Skip-gram with negative sampling (SGNS) B word2vec's method: a logistic classifier, $\sigma(c \cdot w)$, learns to tell real (target, context) pairs from random ones. The learned weights become the embeddings.

Softmax C $e^{z_i} / \Sigma_j e^{z_j}$: turns logits into positive probabilities that sum to 1, keeping their order.

Softmax regression C Multinomial logistic regression, $\hat{y} = \text{softmax}(Wx + b)$. As a language model, W has one row of weights per vocabulary token.

Static embedding B One fixed vector per word type regardless of context (word2vec, GloVe). Can't distinguish the meanings of "it".

Steering vector E The difference between the average hidden states of contrasting prompts, added ($\times \alpha$) to the hidden states at one layer to shift behavior.

Subword A A piece of a word (e.g. "ization") that is a token in its own right.

Subword tokenization A Splitting text into frequent words and word pieces (BPE, WordPiece, Unigram), so rare and unseen words become sequences of known pieces.

Sycophancy E A tendency to agree with the user too easily, which preference training can encourage if raters reward agreement.

System prompt E Instructions silently prepended to a conversation by the app or developer, which set the role, rules and style.

T

tanh C Hyperbolic tangent activation, range $(-1, 1)$. Smooth, but it saturates like sigmoid.

Temperature (τ) E Logits are divided by τ before softmax. Low τ is sharper (closer to greedy); high τ is flatter (more random).

Token A A unit of text produced by a tokenizer: a word, subword, character or byte sequence.

Token ID A The integer index of a token in the vocabulary, which is also its row in the embedding matrix.

Token learner A The training half of a subword tokenizer: it induces the vocabulary from a corpus.

Token segmenter A The applying half of a subword tokenizer: it splits new text using the learned vocabulary and merges.

Tokenizer A An algorithm that splits text into tokens from a fixed vocabulary and maps them to IDs.

Top-k sampling E Keep only the k most probable tokens, renormalize, then sample.

Top-p sampling E Keep the smallest set of tokens whose probabilities sum to at least p, renormalize, then sample. Adapts to the model's confidence.

Training loss E Loss measured on the data the model is trained on.

Transformer block D LayerNorm → multi-head attention → add, then LayerNorm → FFN → add. Blocks are stacked; input and output are both N × d.

Type (word type) A A distinct element of the vocabulary. "the" appearing three times counts as 1 type and 3 instances.

U

Underfitting E Both training and validation loss stay high: the model is too simple or not trained enough.

Unembedding D The d × |V| matrix (often E^T) that maps a hidden state to one logit per vocabulary token.

Unicode A Standard that gives every character in every script (150,000+) a unique code point.

[UNK] token A The single "unknown" token that word-level tokenizers use for every out-of-vocabulary word.

UTF-8 A Variable-length Unicode encoding: 1 byte for ASCII, 2–4 bytes for other characters. Most of the web uses it.

V

Validation loss E Loss on held-out data the model never trains on. It estimates generalization and is used for early stopping.

Value D $v_j = x_j W^V$: the content a token contributes. Values are weighted by attention and summed into the output.

Vanishing gradient C Gradients shrinking toward 0 through saturated sigmoid or tanh units, which stalls learning. ReLU mitigates it.

Vocabulary (V) A The set of all distinct tokens a tokenizer can output.

Vocabulary size (|V|) A The number of tokens in the vocabulary. A trade-off: too big gives rare tokens and large layers; too small gives long sequences.

W

Word embedding B A dense vector representing a word. Similar words have similar vectors.

word2vec B Popular, fast static-embedding method (skip-gram or CBOW) that predicts context words rather than counting them.

WordPiece A BERT's subword algorithm. Like BPE, but it chooses merges by likelihood gain and marks word-internal pieces with ##.

X

XOR C Exclusive OR, a function that isn't linearly separable. It needs a hidden layer with a nonlinearity.

Z

Zero-shot E A prompt with an instruction but no examples.

AI USE STATEMENT

I used Claude Code, Anthropic's AI coding assistant, to help create this study guide. I gave it the Spring 2026 midterm and my six IST 736 lecture decks (Weeks 2–7), and asked it to organize the material into the five topic areas, write a step-by-step worked example for each calculation type, highlight the key terms, and add a glossary. It extracted the slide text, checked every calculation with code, confirmed the GPT-4o token IDs in Q4 by running the o200k_base tokenizer, and produced this PDF. The explanations and midterm answers were checked against the lecture slides. Content that goes beyond the slides, such as the details of WordPiece and the exact form of the KL penalty, is labeled "Beyond the slides".

Sources: IST 736 lecture slides for Weeks 2–7 (tokenization; embeddings; neural network basics; transformers and attention; shaping LLM behavior through training and through prompting) and the Spring 2026 midterm, cited as Q1–Q30.